System Verification and Validation Plan for Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

April 6, 2026

Revision History

Name(s)	Date	Notes
Maham, Cieran and Eman	October 27th, 2025	Added initial first draft of the VnV Plan
Vaisnavi S.	April 5th, 2026	Applied feedback to the VnV Plan
Vaisnavi S.	April 6th, 2026	Finalized the final version of the VnV Plan

Contents

List of Tables

1 Symbols, Abbreviations, and Acronyms

symbol	description
VnV	Verification and Validation
SRS	Software Requirements Specification
API	Application Programming Interface
TC	Test Case
UT	Usability Test
PR	Performance Test
RFT	Reliability and Fault-Tolerance Test
SR	Security Test
MS	Maintainability and Support Test
FR	Functional Requirement
SR-AR	Security Access Requirement

Refer to Section 4 in the SRS document for a table of many more symbols, abbreviations, acronyms, and definitions.

This document aims to outline the verification and validation that the Physiotherapy Companion app will go through during development. The document begins with a brief description as to what the software is, what it is meant to achieve, and some relevant information before explaining in detail the standards and metrics that will be used to evaluate the system.

2 General Information

2.1 Summary

The software focused on in this document is the Physiotherapy Companion application, built for smart devices with camera access to help guide a patient undergoing physiotherapy with their assigned physiotherapy exercises.

The application will provide support and guidance for the exercise and provide real-time feedback to help the user using a video-feed. It will also provide overviews of previous exercises to allow users to see progress and consistency, assisting in forming healthy habits towards recovery and reinforce confidence.

The application will also support a more administrative function for physiotherapists, allowing them to view exercise summaries for patients assigned to them. Additionally, physiotherapists may leave comments on summaries to ensure consistent and timely usage of the application for healthy exercising.

2.2 Objectives

The objectives of the testing expanded upon within this document focus on general system usability, the security of user profiles and video-feed data, and the verification of stakeholder requirements.

Usage of the system should be easily explainable for first-time users with simple dialogue popups and limited visible guiding from one function to the next. Additionally, for the video-feed portion of the application's usage, the user should be reasonably guided through the exercise and process to set them up for successful exercise execution and feedback recommendation acceptance.

User profile and video-feed data should be securely stored, whether locally or within a remote database accessed by the system. User data security has

become an increasingly sensitive topic within the software industry and users may question how their data is being stored and utilized [SecureFlag, 2026]. Users should be aware of the usage of their data and the system should ensure unauthorized access outside of the communicated plan is never possible.

Meeting stakeholder requirements, whether they be data restrictive or process flow in nature, is what will validate the core features of the system. If users do not find value in the expected usage expected from the system, then, ultimately, the system has failed to perform its primary goal. Ensuring stakeholder requirements and feedback are incorporated into the system as it is developed, both initially and onward, will ensure the system is both wanted and usable to the intended audience.

Some out-of-scope objectives include performance and user-interface quality. In this sense, quality means the overall refinement and how polished the interface looks. The implementation should not inhibit usability, but the design need not be too meticulous and visually pleasing.

For performance, the system should respond in a timely manner that does not frustrate a user's ability to perform the main activities of the system. Designing an optimized system which may use minimal amounts of resources is not one of the current objectives of the system. These things could both be expanded into and developed, but for now are considered out of scope.

2.3 Extras

The extras set out for the system consist of both a Design Thinking Document and a Usability Report. The Design Thinking Document aims to explain the process behind, and work done towards, the final design of the system. This document explains the overall flow of processing through the system and the interfacing elements, external libraries utilized, and more. The document flow is iterative, as is the design process, and aims to communicate the entire design process of the system. The Design Thinking Document can be found here. The Usability Report aims to focus on evaluating the system from a user perspective. This report documents usability testing conducted with participants to assess ease of navigation, clarity of instructions, and overall user experience. It includes observations, user feedback, and analysis of interaction patterns to identify the key areas to foucs on and address. The findings are used to ensure the system meets the non-functional requirements and to guide improvements to the interface for both patients and physiotherapists. The Usability Report can be found here.

2.4 Relevant Documentation

- Problem Statement and Goals: Ashraf et al. [2025d]
- Development Plan: Ashraf et al. [2025b]
- System Requirements Specification: Ashraf et al. [2025e]
- Hazard Analysis: Ashraf et al. [2025c]
- Usability Report: Ashraf et al. [2025f]
- Design Thinking Document: Ashraf et al. [2025a]

All of the above document are important artifacts for the implementation of the system. These documents outline how the implementation will be approached and what the implementation will follow.

More documents to come here for external libraries such as OpenCV, pytest, Python Extension, and more when it is determined which libraries and external software will be used.

3 Plan

This section will focus on the methods to verify and validate artifacts of the project. The methods for each facet of the system will be listed below, and additional information including possible questionnaire questions can be found in the appendix at the end of this document. The section will start primarily with verification and move to validation at the end.

3.1 Verification and Validation Team

See *Table 1: Verification and Validation Team*.

3.2 SRS Verification

To verify the SRS, several approaches will be used across different sections of the document. In general, the document will receive verification through feedback from 4G06 teaching assistants, peer-review feedback from another design team, and a check in with a project advisor to ensure the document

Role	Member(s)	Description
Test Development	Maham, Eman	Development of tests for the system and system units based on functional and non-functional requirements listed within the SRS document.
Test Implementation	Cieran	Implementation of a test suite for feasibly software testable requirements, developed based on the tests made by the test development team. Also assurance of software coverage by the suite.
Test Execution	Matthew	Running and verification of the test suite.
Requirements Verification	Vaisnavi	Verification and tracking of requirements to ensure each has been met and tested effectively.
Stakeholder Validation	All	Validation with stakeholders and external users to ensure the system developed performs as users expect; focus on usability.

Table 1: Verification and Validation Team

represents the correct motivation for the project. This check-in will be a self review by the advisor followed by a scheduled meeting to present any probable modifications to the requirements. Based on feedback from the advisor, issues will be created and handled.

To verify the functional requirements within the SRS, the above will be used in addition to software checks for functionality and a requirements checklist which will be updated upon checking functional requirements within the implementation. For verification of non-functional requirements within the SRS, in addition to the general checks, there will be a questionnaire of the software's usage and another checklist with a more flexible scaling system for the NFRs.

3.3 Design Verification

Verification of the design will rely on feedback from 4G06 teaching assistants, peer-review feedback provided by colleagues on another design team, as well as advisor and stakeholder inquiries. The feedback from teaching assistants and peer-review feedback will be created as issues within our repository and handled accordingly. For advisor and stakeholder inquiry, there will be software handouts along with a questionnaire which is expanded on more in our Usability Report, which is referenced in the appendix of this document (Section 6.1).

3.4 Verification and Validation Plan Verification

This document will also be reviewed by the teaching assistants and provided peer-review by another design team, creating issues within the GitHub repository to be dealt with accordingly. To ensure feedback is properly actioned, the following step-by-step process is followed:

- Feedback is collected during TA and peer review sessions
- Each comment is converted into a GitHub issue
- Issues are assigned to a responsible team member
- Changes are implemented and committed with reference to the issue
- Updated sections are reviewed again before finalizing

An issue is considered resolved once the required changes have been implemented, reviewed by at least one additional team member, and the corresponding GitHub issue has been closed. In addition, the test suite derived from this document will be verified via code coverage on the software. For validation, the final outcome of each test will be discussed in the meeting with the project advisor to ensure the system is doing what it should in each test case.

3.5 Implementation Verification

The implementation verification plan ensures that the system aligns with the System Requirements Specification (SRS) through both dynamic and non-dynamic testing methods.

Dynamic testing is conducted through unit testing, integration testing, and system-level validation. Unit tests are implemented for both the frontend and backend components to verify core functionality such as authentication, data handling, and computer vision analysis. Integration testing is used to validate interactions between system components, including Firebase authentication, Firestore database operations, and backend API endpoints such as `/process_frame` and `/precheck_frame`. Performance testing is also carried out to evaluate response time during exercise recording (≤ 2 seconds), system stability under repeated usage, and handling of real-time data processing.

Static (non-dynamic) testing is used to evaluate code and design quality without executing the system. This includes peer code reviews on all pull requests to ensure best practices are followed. Code walkthroughs are also conducted for critical modules and functionalities to identify potential issues early. In addition, tools like `pylint` are used to catch code quality issues such as inconsistent formatting and unused variables. Documentation is regularly reviewed, with issues tracked and resolved through GitHub.

Automated test suites and GitHub Actions are used to ensure functionality remains intact and to help maintain system stability. Manual testing is also carried out for real-time features, including MediaPipe pose detection and user interaction workflows that cannot be fully automated. These combined efforts help ensure that all features and functionality remain reliable.

3.6 Automated Testing and Verification Tools

Unit Testing: The project uses `pytest` for the Python backend to validate API endpoints, data processing, and computer vision functionality. For the frontend, `Jest` is used to verify the UI and component interactions.

Code Coverage: Code coverage is used to measure how much of the system is covered by the test cases.

- **Python:** We use `pytest-cov` with `pytest` to keep track of code coverage for the backend.
- **TypeScript:** We use `Jest` to keep track fo code coverage for the frontend.

Overall, the goal is to achieve meaningful code coverage, and identify any gaps so we know where to focus next. To maintain code quality, we use static analysis tools such as `pylint` to help with code quality where applicable. In addition, we conduct peer code reviews and walkthroughs consistently to ensure our code is abiding by best practices. In terms of automated workflows, we use GitHub Actions to conduct automated builds and checks on pull requests. Ultimately, these workflows help ensure that the project remains stable as new changes to the code are constantly being introduced.

3.7 Software Validation

For software validation, to ensure the correct system was designed to solve the problems outlined within the problem statement document, there will be system walkthroughs and interviews with stakeholders and the project advisor. These interviews will assess time to use the systems functions, understanding of the system and how it processes data, as well as a general acceptance of the system. The interview will end with questions outlined in our Usability Report which is referenced in the appendix of this document (Section 6.1).

4 System testing

This section outlines the tests used to verify both the functional requirements (FRs) and non-functional requirements (NFRs) of the system. Ultimately,

these tests ensure that the software behaves as expected in terms of its core functionality, while also meeting performance, usability, and reliability expectations.

4.1 Tests for Functional Requirements

1. **test-TC1 : Physiotherapist Registration (Valid License)**

   Control: Automated

   Initial State: App launched; registration screen accessible; Firebase Auth and Firestore active.

   Input: `registerPhysio()` called with a valid license format and an active license in `validLicenses`.

   Output: Auth user created. Firestore `users/{uid}` written with `role="physio"`, `verified=true`, and `createdAt`. All assertions passed.

   Test Case Derivation: Derived from requirement FR1.1 and FR1.2 in the SRS, which state that verified physiotherapists must be able to register successfully using a valid license.

   Pass/Fail Criteria: Pass if the account is created successfully and the correct Firestore fields are written. Fail if registration is rejected or the stored data is incomplete or incorrect.

   How test will be performed: Call `registerPhysio()` with a valid and active license, then verify that authentication succeeds and the correct physiotherapist profile is created in Firestore.

2. **test-TC2 : Physiotherapist Registration (Invalid License Format)**

   Control: Automated

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with an invalid license format (missing dash or incorrect digits).

   Output: Error "Invalid license format" thrown. No Auth user or Firestore writes created.

Test Case Derivation: Derived from requirement FR1.1 and FR1.2 in the SRS, which require that the system validates input format and rejects invalid physiotherapist license formats.

Pass/Fail Criteria: Pass if the invalid license is rejected, the correct error is thrown, and no account data is created. Fail otherwise.

How test will be performed: Call `registerPhysio()` with incorrectly formatted license values and verify that registration fails and no user data is written to Firestore.

3. **test-TC3 : Physiotherapist Registration (Unverified License)**

   Control: Automated

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with valid format but license not found or `active=false` in `validLicenses`.

   Output: Error "License not verified" thrown. No Auth user or Firestore writes created.

   Test Case Derivation: Derived from requirement FR1.1 and FR1.2 in the SRS, which specify that only verified physiotherapists are allowed to create accounts.

   Pass/Fail Criteria: Pass if the unverified license is rejected, the correct error is returned, and no account is created. Fail otherwise.

   How test will be performed: Call `registerPhysio()` with a correctly formatted but unverified license and confirm that the system prevents registration and does not write profile data.

4. **test-TC4 : Patient Registration (Valid Invite Code)**

   Control: Automated

   Initial State: App launched; `inviteCodes` collection seeded with an active, unused code containing a `physioId`.

   Input: `registerPatient()` called with a valid invite code.

   Output: Auth user created. Firestore `users/{uid}` written with `role="patient"` and stored `physioId`. Invite code updated with `used=true` and `usedBy={uid}`.

Test Case Derivation: Derived from requirements FR1.1, FR1.3, FR2, and FR3 in the SRS, which state that patients must register using valid invite codes and be linked to a physiotherapist.

Pass/Fail Criteria: Pass if the patient account is created, the linked physiotherapist is stored, and the invite code is marked as used. Fail otherwise.

How test will be performed: Call `registerPatient()` with a valid invite code and verify account creation, Firestore data, and invite code update behaviour.

5. **test-TC5 : Patient Registration (Non-Existent Invite Code)**

   Control: Automated

   Initial State: App launched; `inviteCodes` collection does not contain the submitted code.

   Input: `registerPatient()` called with a code that does not exist in `inviteCodes`.

   Output: Error "Invalid invite code" thrown. No Auth user or Firestore writes created.

   Test Case Derivation: Derived from requirements FR1.1 and FR1.3 in the SRS, which require that the system rejects invalid or non-existent invite codes during patient registration.

   Pass/Fail Criteria: Pass if the invalid code is rejected and no user record is created. Fail otherwise.

   How test will be performed: Call `registerPatient()` with a code that is not present in the invite code collection and verify that registration is blocked.

6. **test-TC6 : Patient Registration (Inactive Invite Code)**

   Control: Automated

   Initial State: App launched; `inviteCodes` contains a matching code with `active=false`.

   Input: `registerPatient()` called with an inactive invite code.

   Output: Error "Invite code inactive" thrown. No Auth user or Firestore writes created.

Test Case Derivation: Derived from requirements FR1.1 and FR1.3 in the SRS, which state that inactive invite codes must not be accepted for patient registration.

Pass/Fail Criteria: Pass if the inactive code is rejected and no account is created. Fail otherwise.

How test will be performed: Call `registerPatient()` with an inactive invite code and verify that the request fails and no database writes occur.

7. **test-TC7 : Patient Registration (Already Used Invite Code)**

   Control: Automated

   Initial State: App launched; `inviteCodes` contains a matching code with `used=true`.

   Input: `registerPatient()` called with an already used invite code.

   Output: Error "Invite code already used" thrown. No Auth user or Firestore writes created.

   Test Case Derivation: Derived from requirements FR1.1 and FR1.3 in the SRS, which enforce that invite codes are single-use and cannot be reused.

   Pass/Fail Criteria: Pass if reuse of the invite code is blocked and no new account is created. Fail otherwise.

   How test will be performed: Call `registerPatient()` using a code already marked as used and verify that the system rejects the request.

8. **test-TC8 : Patient Registration (Invite Code Missing Physio Link)**

   Control: Automated

   Initial State: App launched; `inviteCodes` contains a matching code with no `physioId` field.

   Input: `registerPatient()` called with a code missing a `physioId`.

   Output: Error "Invite code missing physio link" thrown. No Auth user or Firestore writes created.

Test Case Derivation: Derived from requirements FR1.1, FR1.3, and FR2 in the SRS, which require that patient accounts must be linked to a physiotherapist through a valid invite code.

Pass/Fail Criteria: Pass if the missing link case is rejected and no account data is created. Fail otherwise.

How test will be performed: Call `registerPatient()` with an invite code that lacks a `physioId` and verify that the system throws the correct error and rejects registration.

9. **test-TC9 : User Login (Valid Physiotherapist Credentials)**

   Control: Automated

   Initial State: Login screen available; Firestore contains a profile with `role="physio"`.

   Input: `loginUser()` called with valid PT credentials.

   Output: Login returns object with `uid` and profile data including `role="physio"`. No errors thrown.

   Test Case Derivation: Derived from requirements FR1 and FR1.1 in the SRS, which state that registered physiotherapists must be able to log in using valid credentials.

   Pass/Fail Criteria: Pass if login succeeds and the correct physiotherapist profile data is returned. Fail otherwise.

   How test will be performed: Call `loginUser()` with valid physiotherapist credentials and verify that authentication succeeds and the returned user object contains the expected role and profile data.

10. **test-TC10 : User Login (Valid Patient Credentials)**

    Control: Automated

    Initial State: Login screen available; Firestore contains a profile with `role="patient"`.

    Input: `loginUser()` called with valid patient credentials.

    Output: Login returns object with `uid` and profile data including `role="patient"`. No errors thrown.

Test Case Derivation: Derived from requirements FR1 and FR1.1 in the SRS, which state that registered patients must be able to log in using valid credentials.

Pass/Fail Criteria: Pass if login succeeds and the correct patient profile is returned. Fail otherwise.

How test will be performed: Call `loginUser()` with valid patient credentials and verify that authentication and profile retrieval both succeed.

11. **test-TC11 : User Login (Missing Firestore Profile)**

    Control: Automated

    Initial State: Login screen available; valid Firebase Auth credentials exist but no Firestore profile.

    Input: `loginUser()` called with valid credentials.

    Output: Error "User profile missing in Firestore" thrown. Login rejected.

    Test Case Derivation: Derived from requirements FR1 and FR1.1 in the SRS, which require that login depends on both valid authentication credentials and a valid user profile.

    Pass/Fail Criteria: Pass if login is rejected and the correct error is returned when the Firestore profile is missing. Fail otherwise.

    How test will be performed: Call `loginUser()` for a valid auth account with no corresponding Firestore profile and confirm that login fails as expected.

12. **test-TC12 : User Logout**

    Control: Automated

    Initial State: User session active.

    Input: `logoutUser()` called.

    Output: Auth sign-out called once and completes without error.

    Test Case Derivation: Derived from requirement FR1 in the SRS, which states that users must be able to securely log out and end their session.

Pass/Fail Criteria: Pass if the user is signed out successfully and no error occurs. Fail otherwise.

How test will be performed: Call `logoutUser()` while a user session is active and verify that sign-out completes successfully.

13. **test-TC13 : Get User Data**

    Control: Automated

    Initial State: Firestore populated with test user documents.

    Input: `getUserData()` called with a valid uid, a non-existent uid, and an empty uid.

    Output: Valid uid returns `UserData`. Non-existent uid returns null. Empty uid throws an error.

    Test Case Derivation: Derived from requirement FR21 in the SRS, which states that the system must retrieve and provide user data correctly.

    Pass/Fail Criteria: Pass if valid uid returns correct data, non-existent uid returns null, and empty uid throws an error. Fail otherwise.

    How test will be performed: Call `getUserData()` with different uid cases and verify returned values and error handling.

14. **test-TC14 : User Lookup by Email**

    Control: Automated

    Initial State: Firestore populated with test user documents.

    Input: `getUserByEmail()` called with a registered email and an unregistered email.

    Output: Registered email returns user object and true. Unregistered email returns null and false.

    Test Case Derivation: Derived from requirement FR21 in the SRS, which specifies that users must be searchable and retrievable by identifying information such as email.

    Pass/Fail Criteria: Pass if valid email returns correct user and invalid email returns null. Fail otherwise.

How test will be performed: Call `getUserByEmail()` with both valid and invalid inputs and verify results.

15. **test-TC15 : Update User Data**

    Control: Automated

    Initial State: Firestore populated with test user documents.

    Input: `updateUser()` called with a valid update payload.

    Output: Update returns true. Changes reflected in Firestore.

    Test Case Derivation: Derived from requirements FR2, FR6, and FR22 in the SRS, which state that users must be able to update their stored profile information.

    Pass/Fail Criteria: Pass if update returns true and Firestore reflects changes. Fail otherwise.

    How test will be performed: Call `updateUser()` and verify updated values in Firestore.

16. **test-TC16 : Delete User**

    Control: Automated

    Initial State: Firestore populated with test user documents.

    Input: `deleteUser()` called with a valid uid, a valid email, and a non-existent case.

    Output: Valid cases return true. Non-existent case returns false.

    Test Case Derivation: Derived from requirement FR4 in the SRS, which states that users must be removable from the system.

    Pass/Fail Criteria: Pass if valid users are deleted and invalid cases return false. Fail otherwise.

    How test will be performed: Call `deleteUser()` with different inputs and verify outcomes.

17. **test-TC17 : User Queries**

    Control: Automated

    Initial State: Firestore populated with linked PT and patient records.

Input: `getPatientsByPhysioId()`, `searchByName()`, and `getAllUsers()` called.

Output: Each function returns correctly filtered arrays. Errors return empty arrays.

Test Case Derivation: Derived from requirement FR18 in the SRS, which requires that the system support querying and filtering of user data.

Pass/Fail Criteria: Pass if queries return correct filtered results. Fail otherwise.

How test will be performed: Call each query function and verify returned datasets.

18. **test-TC18 : User Info Lookup**

    Control: Automated

    Initial State: Firestore populated with linked PT and patient records.

    Input: `getUserdbInfo()` called with name only and with name and birthday filter.

    Output: Returns categorized patients and physiotherapists correctly. Birthday filter works as expected.

    Test Case Derivation: Derived from requirements FR21 and FR25 in the SRS, which specify that the system must support detailed user lookup and filtering.

    Pass/Fail Criteria: Pass if results are correctly categorized and filtered. Fail otherwise.

    How test will be performed: Call function with different filters and verify output.

19. **test-TC19 : PT Account Delete**

    Control: Automated

    Initial State: Firestore populated with a valid physiotherapist and linked patient.

    Input: `deletePTAccount()` called with a valid physio, a non-physio user, and a non-existent patient.

Output: Valid case deletes patient successfully. Non-physio and non-existent cases throw appropriate errors.

Test Case Derivation: Derived from requirements FR2, FR4, and FR22 in the SRS, which require that physiotherapists can manage and delete linked patient accounts.

Pass/Fail Criteria: Pass if valid deletions succeed and invalid cases throw errors. Fail otherwise.

How test will be performed: Call function with different user types and verify behaviour.

20. **test-TC20 : User Authentication Validation**

    Control: Automated

    Initial State: Firestore populated with test user credentials.

    Input: `usernamePwMatch()` and `authenticateUser()` called with valid credentials, an invalid password, and a non-existent user.

    Output: Valid credentials return true and user object. Invalid password throws an error. Non-existent user returns null.

    Derived from requirement FR1 and FR1.1 in the SRS, which state that users must be authenticated using valid credentials before accessing the system.

    Pass/Fail Criteria: Pass if valid credentials succeed and invalid ones fail appropriately. Fail otherwise.

    How test will be performed: Call authentication functions with different credential sets and verify responses.

21. **test-TC21 : System Initialization**

    Control: Automated

    Initial State: System not yet initialized.

    Input: `initializeSystem()` called.

    Output: Success message logged to console. No errors thrown.

    Test Case Derivation: Derived from requirement FR1 in the SRS, which implies that the system must initialize correctly to support user authentication and access to core functionality.

Pass/Fail Criteria: Pass if initialization completes without errors. Fail otherwise.

How test will be performed: Call initialization function and verify no errors occur.

22. **test-TC22 : Credential Validation**

    Control: Automated

    Initial State: User account exists in Firestore.

    Input: `validateCredentials()` called with a correct password and an incorrect password.

    Output: Correct password returns true. Incorrect password returns false.

    Test Case Derivation: Derived from requirement FR1 and FR1.1 in the SRS, which require that user credentials must be validated correctly during authentication.

    Pass/Fail Criteria: Pass if correct password succeeds and incorrect fails. Fail otherwise.

    How test will be performed: Call function with valid and invalid passwords and verify output.

4.2 Tests for Nonfunctional Requirements

4.2.1 Usability

1. **UT-1 Navigation Clarity**

   Type: User Testing
   Requirements Covered: LFR-AP1
   Initial State: App installed on a mobile device; user logged in.
   Input/Condition: User is asked to find and open the exercise screen, start a recording session, and return to the dashboard without assistance.
   Output/Result: User successfully navigates through the required screens without confusion. Labels and buttons are reported as clear.
   How test will be performed: A participant will be observed completing navigation tasks independently and their feedback will be recorded.

2. **UT-2 Instruction Clarity During Exercise**

   Type: User Testing
   Requirements Covered: UHR-EOU2, UHR-LRN1
   Initial State: User is on the record exercise screen with camera access enabled.
   Input/Condition: User initiates an exercise session and follows on-screen feedback prompts.
   Output/Result: Instructions are reported as clear and aligned with required actions. Minor wording improvements are identified and updated.
   How test will be performed: A participant will perform an exercise session and provide feedback on clarity of instructions.

3. **UT-3 Survey Questionnaire – ui_testing**

   Type: Survey Questionnaire
   Requirements Covered: UHR-EOU1, LFR-ST1
   Initial State: App accessible to testers on mobile devices.
   Input/Condition: Five users complete a usability questionnaire evaluating ease of use, clarity, and navigation.
   Output/Result: Average rating across categories is $\geq 4/5$. Minor UI refinements are identified and implemented.
   How test will be performed: Users interact with the application and complete a structured usability survey.
   Quantifiable Metric: Average usability score must be $\geq 4/5$.

4. **UT-4 Account Creation (Usability)**

   Type: User Testing
   Requirements Covered: UHR-EOU1, UHR-LRN1
   Initial State: App installed; invite code and PT license provided.
   Input/Condition: User completes account creation flow.
   Output/Result: Users successfully create accounts without assistance. Fields are completed without confusion.
   How test will be performed: A participant will complete account setup while being observed.

5. **UT-5 Exercise Recording Initiation**

   Type: User Testing
   Requirements Covered: UHR-EOU2, UHR-AR1
   Initial State: Patient logged in; camera permission granted; backend running.
   Input/Condition: User navigates to Record screen and starts recording.
   Output/Result: Camera activates successfully and user follows prompts without confusion.
   How test will be performed: A participant will initiate recording while being observed.

6. **UT-6 Real-Time Pose Detection and Feedback**

   Type: User Testing
   Requirements Covered: UHR-UPL1, UHR-AR1
   Initial State: User positioned in frame; pose detection active.
   Input/Condition: User performs exercise repetitions.
   Output/Result: Feedback overlays are understandable and rep. counts are perceived as accurate.
   How test will be performed: A participant performs exercises and evaluates feedback clarity.

7. **UT-7 Exercise Page (Instructions)**

   Type: User Testing
   Requirements Covered: UHR-LRN1, UHR-EOU1
   Initial State: Exercises tab accessible.
   Input/Condition: User reviews exercise instructions.
   Output/Result: Instructions are clear and easy to follow.
   How test will be performed: A participant reads instructions and explains them back.

8. **UT-8 Progress Screen (Session History)**

   Type: User Testing
   Requirements Covered: UHR-EOU1, LFR-ST1

Initial State: User has recorded session data.
Input/Condition: User views progress screen.
Output/Result: Data is readable and easy to interpret.
How test will be performed: A participant reviews the screen and explains insights.

9. **UT-9 PT Dashboard (Patient Activity)**

   Type: User Testing
   Requirements Covered: UHR-EOU1, LFR-AP1
   Initial State: Physiotherapist logged in with patient data.
   Input/Condition: PT reviews dashboard and selects a patient.
   Output/Result: Dashboard is easy to navigate and patient data is clear.
   How test will be performed: A physiotherapist interacts with dashboard features.

10. **UT-10 PT Feedback Submission**

    Type: User Testing
    Requirements Covered: UHR-EOU1, UHR-UPL1
    Initial State: PT viewing patient session details.
    Input/Condition: PT submits written feedback.
    Output/Result: Feedback is successfully saved and visible to patient.
    How test will be performed: A physiotherapist submits feedback and verifies it appears correctly.

4.2.2 Performance

1. **PR-1 Exercise Interface Response Time – ex_int_speed**

   Type: Manual, Dynamic
   Requirements Covered: PR-SL1, PR-SL2
   Initial State: User logged in on a mobile device; backend server running.
   Input/Condition: User starts recording from the record screen. Screen recording timestamps are used to measure time from tapping "Start"

to first visible feedback. Five trials are conducted.
Output/Result: Response times recorded as 1.87s, 1.92s, 1.79s, 1.95s, and 1.84s. Average response time is 1.87 seconds ($\leq$ 2 seconds).
How test will be performed: Tester records screen during interaction and calculates time differences across trials.

2. **PR-2 Repeated Use Performance**

   Type: Manual, Dynamic
   Requirements Covered: PR-SL1
   Initial State: Application running normally on mobile device.
   Input/Condition: User completes 10 consecutive "Start $\rightarrow$ Record $\rightarrow$ Stop" cycles.
   Output/Result: No slowdown observed. All response times remain within acceptable range and no crashes occur.
   How test will be performed: Tester performs repeated cycles and monitors responsiveness and stability.

4.2.3 Reliability and Fault-Tolerance

1. **RFT-1 Backend Failure Handling – fail_recovery**

   Type: Manual, Dynamic
   Requirements Covered: PR-RFT1
   Initial State: User actively using application connected to backend.
   Input/Condition: Backend server is manually terminated during active analysis.
   Output/Result: Application displays error message and remains responsive. After restart, user continues normally.
   How test will be performed: Tester shuts down backend during execution and observes system behaviour.

2. **RFT-2 Network Interruption During Upload – Interrupted_vid**

   Type: Manual, Dynamic
   Requirements Covered: PR-RFT2
   Initial State: User uploading recorded exercise video.

Input/Condition: WiFi is disabled mid-upload and restored after 10 seconds.
Output/Result: Upload failure detected and retry option shown. Retry succeeds and data remains intact.
How test will be performed: Tester disables network during upload and verifies recovery behaviour.

4.2.4 Security

1. **SR-1 Unauthorized Access to Protected Endpoint**

   Type: Manual, Dynamic
   Requirements Covered: SR-AR1
   Initial State: Backend server running with authentication enabled.
   Input/Condition: Request sent to protected endpoint without authentication.
   Output/Result: Server returns HTTP 401 Unauthorized and no protected data is exposed.
   How test will be performed: Tester sends unauthenticated API request using tools like Postman.

2. **SR-2 Invalid Credentials Login**

   Type: Manual, Dynamic
   Requirements Covered: SR-AR1
   Initial State: Login screen available.
   Input/Condition: User attempts login with invalid credentials.
   Output/Result: Login denied with appropriate error message and no system data exposed.
   How test will be performed: Tester inputs invalid credentials and observes system response.

4.2.5 Maintainability and Support

1. **MS-1 Reproducible Setup**

Type: Manual, Dynamic
Requirements Covered: OER-WE1, OER-PR1
Initial State: Fresh machine with no prior setup.
Input/Condition: Repository is cloned and setup instructions are followed.
Output/Result: Application runs successfully with no missing dependencies.
How test will be performed: Tester follows README setup steps and verifies system runs correctly.

2. **MS-2 Code Review and Quality Assurance**

Type: Static Analysis and Manual Review
Requirements Covered: N/A
Initial State: Revision 0 completed.
Input/Condition: Peer reviews, unit tests and usability tests are conducted.
Output/Result: Code passes tests, issues are resolved, and the overall code structure remains modular and maintainable.
How test will be performed: Reviewers analyze pull requests, run tests, and verify code quality and usability improvements.

4.3 Traceability Between Test Cases and Requirements

Table 2: Functional Requirements and Corresponding Test Cases

Test Section	**Functional Requirement(s)**
Physiotherapist Registration Tests (TC1–TC3)	FR1.1, FR1.2
Patient Registration Tests (TC4–TC8)	FR1.1, FR1.3, FR2, FR3
User Authentication and Session Tests (TC9–TC12, TC20, TC22)	FR1, FR1.1
User Data Management Tests (TC13–TC16)	FR2, FR4, FR6, FR21, FR22
User Query and Lookup Tests (TC17–TC18)	FR18, FR21, FR25
Physiotherapist Account Management Tests (TC19)	FR2, FR4, FR22
System Initialization Tests (TC21)	FR1

Table 3: Usability Tests and Corresponding Requirements

Test ID	**Non-Functional Requirement**
UT-1	LFR-AP1
UT-2	UHR-EOU2, UHR-LRN1
UT-3	UHR-EOU1, LFR-ST1
UT-4	UHR-EOU1, UHR-LRN1
UT-5	UHR-EOU2, UHR-AR1
UT-6	UHR-UPL1, UHR-AR1
UT-7	UHR-LRN1, UHR-EOU1
UT-8	UHR-EOU1, LFR-ST1
UT-9	UHR-EOU1, LFR-AP1
UT-10	UHR-EOU1, UHR-UPL1

Table 4: Performance Tests and Corresponding Requirements

Test ID	**Non-Functional Requirement**
PR-1	PR-SL1, PR-SL2
PR-2	PR-SL1

Table 5: Reliability & Fault-Tolerance Tests and Corresponding Requirements

Test ID	**Non-Functional Requirement**
RFT-1	PR-RFT1
RFT-2	PR-RFT2

Table 6: Security Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
SR-1	SR-AR1
SR-2	SR-AR1

Table 7: Maintainability & Support Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
MS-1	OER-WE1, OER-PR1

5 Unit Test Description

This section outlines the unit testing approach and scope for each of the modules. Please note that the VnV Report contains the detailed individual test cases and expected results, but this section essentially summarizes the plan for the unit testing plan by module as well as the key functionalities that are covered.

5.1 Authentication

5.1.1 Account Creation Module Tests

Goal: The account creation module verifies that both physiotherapist and patient registration flows behave correctly under valid and invalid conditions. This includes validating physiotherapist license formats, checking license verification status, validating patient invite codes, and ensuring new users are stored correctly in Firebase Authentication and Firestore.

Target requirement(s): FR1.1, FR1.2, FR1.3, FR2, FR3

- **Physiotherapist registration with valid license:**
  1. Ensures a physiotherapist account is created only when the provided license format is valid and verified.
  2. Verifies Firestore user data is written with the correct role, fields, and verification status.

- **Physiotherapist registration failure cases:**
    1. Ensures invalid license formats are rejected with the correct error.
    2. Verifies that unverified or inactive licenses do not allow account creation.
- **Patient registration with valid invite code:**
    1. Ensures a patient can register when the invite code is active, unused, and linked to a physiotherapist.
    2. Verifies the patient profile and invite-code data are stored correctly.
- **Patient registration failure cases:**
    1. Ensures invalid, inactive, or already-used invite codes are rejected.
    2. Verifies registration does not proceed when a required link to a physiotherapist is missing.

5.1.2 Valid User Module Tests

Goal: The valid user module verifies user authentication after account creation. These tests ensure that users with valid credentials can log in successfully, that missing profile information is handled correctly, and that logout functionality successfully clears the current active session.

Target requirement(s): FR1, FR1.1, SR-AR1

- **Valid login behaviour:**
    1. Ensures physiotherapist and patient accounts can log in using valid credentials.
    2. Verifies that profile data is returned correctly after authentication.
- **Login failure handling:**
    1. Ensures login is rejected when the expected Firestore user profile is missing.
- **Logout behaviour:**
    1. Verifies the sign-out operation is called once and completes without error.

5.2 User Account Module

5.2.1 User Management Functions

Goal: The user account module provides database support for retrieving, updating, and deleting user information. These tests verify correct data retrieval, updates, and deletion logic for valid, invalid, and missing user IDs.

Target requirement(s): FR2, FR4, FR6, FR21, FR22, FR23

- **User retrieval:**
  1. Ensures user data can be retrieved using valid uid.
  2. Verifies null or error states for non-existent or invalid inputs.
- **Email-based lookup:**
  1. Ensures users can be found through registered email addresses.
  2. Verifies that unknown emails do not incorrectly return data.
- **User update behaviour:**
  1. Ensures valid user profile updates are written correctly.
  2. Verifies changes are reflected in Firestore after the update operation.
- **User deletion behaviour:**
  1. Ensures valid user deletion requests succeed.
  2. Verifies deletion attempts for non-existent users fail safely.

5.2.2 Query and Lookup Functions

Goal: These tests verify that the module supports user querying, filtering, and specialized lookup operations needed by the application. This includes retrieving linked patients, searching by name, and applying additional filters such as birthday matching.

Target requirement(s): FR18, FR21, FR25

- **General query behaviour:**
  1. Ensures linked patients and search results are filtered correctly.

- **Detailed user lookup:**

    1. Ensures user database lookups return appropriately categorized physiotherapist and patient results.

5.2.3 Specialized and Validation Functions

Goal: These tests verify specialized user-account actions such as physiotherapist-linked account deletion, user authentication functions,and password validation logic.

Target requirement(s): FR1, FR2, FR4, FR22, SR-AR1, SR-AR2, SR-PR1

- **Physiotherapist-linked patient account management:**

    1. Ensures physiotherapists can remove linked patient accounts when appropriate.
    2. Verifies invalid deletion attempts are rejected with the correct errors.

- **Authentication helper validation:**

    1. Ensures username-password matching and authentication utilities behave correctly for valid and invalid credentials.
    2. Verifies non-existent users do not incorrectly authenticate.

- **Initialization and credential validation:**

    1. Ensures system initialization routines complete successfully.

5.3 Live Feedback Module

5.3.1 Corrective Feedback and Rep. Counting

Goal: The live feedback module verifies that the system generates real-time corrective messages and repetition updates based on incoming joint-angle measurements. These tests cover valid range-of-motion behaviour, insufficient movement, excessive movement, and loss of pose visibility.

Target requirement(s): FR13, FR14

- **Valid movement detection:**

    1. Ensures no corrective feedback is generated when joint angles remain within the acceptable range.
    2. Verifies rep. counting increments correctly for valid repetitions.

- **Insufficient or excessive movement:**

    1. Ensures corrective messages are produced when joint angles fall below the expected range.
    2. Verifies the system responds appropriately when movement exceeds defined thresholds.

- **No pose detected:**

    1. Ensures the system provides a clear prompt when the user is not detected in frame.

5.3.2 Latency Handling

Goal: These tests that the repeated feedback generation is responsive during the exercise sequences.

Target requirement(s): FR14, FR16, PR-SL2

- **Repeated feedback latency:**

    1. Verifies multiple valid repetitions continue to receive feedback within the required latency threshold.

5.3.3 Physiotherapist Comment Submission and Retrieval

Goal: These tests verify the ability of physiotherapists to leave feedback comments on patient sessions and the ability of patients to retrieve those comments later.

Target requirement(s): FR19, FR20, FR25

- **Comment submission:**

    1. Ensures physiotherapist comments are saved to the correct session record.

2. Verifies comments are associated with the correct, respective patient activity.

- **Comment retrieval:**

1. Ensures patients can retrieve stored physiotherapist comments for completed sessions.
2. Verifies null is returned appropriately when no comment exists.

5.4 Video Capture and Review Module

5.4.1 Pose Connection and Landmark Extraction Tests

Goal: The video capture and review module verifies the pose connection drawing and landmark extractions used for camera processing and visual overlays. These tests ensure that full-body and leg-only connections are returned correctly and that valid pose results are converted into usable landmark lists.

Target requirement(s): FR12, FR26

- **Pose connection retrieval:**

1. Ensures full pose connections and leg-only connections are returned correctly depending on the selected mode.

- **Landmark extraction:**

1. Ensures valid pose results return the expected landmark lists.
2. Verifies missing or empty pose data returns `None` safely.

5.4.2 Camera Initialization and Frame Processing Tests

Goal: These tests verify the camera and pose detection. This includes initialization, frame capture, pose detection, and the landmark drawing for the live processing.

Target requirement(s): FR7, FR12, FR14

- **Camera initialization:**

1. Ensures the camera is configured with the expected display size and frame properties.

- **Frame retrieval:**
    1. Verifies frames are returned and resized correctly.
- **Pose processing:**
    1. Ensures pose results are generated from valid frames.
    2. Verifies detection is invoked with correctly transformed image inputs.
- **Landmark drawing:**
    1. Ensures landmarks and connections are drawn correctly on returned frames.

5.5 Computer Vision and Form Analysis Module

5.5.1 Angle and Knee-Angle Calculation Tests

Goal: The computer vision and form analysis module verifies that the angle calculations and computed knee-angle behave correctly across valid and invalid conditions.

Target requirement(s): FR12, FR26

- **Angle calculation correctness:**
    1. Ensures known point sets return expected angle values.
    2. Verifies edge cases such as `None` inputs are handled safely.
- **Knee angle extraction:**
    1. Ensures left and right knee angles can be computed correctly from valid pose results.
    2. Verifies missing landmarks and empty pose results do not crash processing.

5.5.2 Calibration and Rep Analysis Tests

Goal: These tests verify the analyzer's calibration process, range-of-motion updates, rep-state transitions, and rep-counting behaviour over time.

Target requirement(s): FR8, FR11, FR12, FR14, FR16, FR17, FR27

- **Calibration flow:**
  1. Ensures calibration begins and ends correctly while tracking min and max observed angles.
  2. Verifies calibration ignores invalid or missing values safely.
- **Rep-state transitions:**
  1. Ensures the analyzer transitions between flexion and extension states correctly.
  2. Verifies valid rep-state transition counts and record durations only when appropriate.
- **Range-of-motion tracking:**
  1. Ensures min and max angles expand correctly as new values arrive.
  2. Verifies that unusual, negative, or out-of-range inputs do not break the analyzer.
- **Summary metrics:**
  1. Ensures summary outputs include expected metrics such as rep. count, ROM, durations, and calibration state.
  2. Verifies metric rounding and empty-duration handling behave correctly.

5.6 Performance and Latency Module

5.6.1 Backend Reset and API Endpoint Tests

Goal: The performance and latency module verifies that the backend services initialize correctly, reset correctly, and respond appropriately to valid and invalid API requests.

Target requirement(s): FR7, FR8, FR12, FR26

- **Backend initialization and reset:**
    1. Ensures the backend starts in the correct state and can be reset safely.
    2. Verifies reset endpoints return successful responses and replace analyzer state correctly.
- **Frame endpoint validation:**
    1. Ensures invalid or missing image data results in the correct error response.
    2. Verifies valid frame requests return landmarks and metrics when pose data is present.
- **Mirroring and side handling:**
    1. Ensures mirrored and non-mirrored inputs are processed appropriately.
    2. Verifies correct side selection is passed into the knee-angle calculation logic.

5.6.2 Frontend Service and Storing Metrics Tests

Goal: These tests verify the frontend service helpers that communicate with the backend and store activity/session metrics to Firestore.

Target requirement(s): FR1, FR1.1, FR7, FR8, FR11, FR17, FR21

- **Frontend request behaviour:**
    1. Ensures frontend service calls send the correct payloads based on camera facing, side, and reset operations.
    2. Verifies network and reset failures are handled correctly.
- **Storing Metrics:**
    1. Ensures metrics are saved only when a valid user is logged in and corresponding Firestore data exists.
    2. Verifies saved activity documents include expected session fields and default values.

- **Session persistence:**
    1. Ensures session summary data is stored correctly for completed exercise sessions.
    2. Verifies the default behaviour when optional metric fields are missing.
- **User display-name lookup:**
    1. Ensures the system retrieves a display name from Firestore when available.
    2. Verifies safe default values are used when the user is missing or lookup fails.

5.7 Profile Activity Module

5.7.1 Current User and Activity Retrieval Tests

Goal: The profile activity module verifies retrieval of current user data, current user IDs, activity histories, and summary information from Firestore.

Target requirement(s): FR10, FR11, FR17, FR18, FR21, FR23

- **Current user retrieval:**
    1. Ensures current user and current user ID can be retrieved correctly.
    2. Verifies null is returned safely when no current user exists or Firestore errors occur.
- **Activity history retrieval:**
    1. Ensures activities for a user are returned correctly when records exist.
    2. Verifies empty arrays are returned when no activity data is found or when an error occurs.
- **Summary Data:**
    1. Ensures summary data is computed correctly from available user activity.
    2. Verifies duplicate dates and missing data are handled safely.

5.7.2 Chart and Comment Data Tests

Goal: These tests verify chart generation, comment retrieval, comment posting, and user-name lookup functions used throughout the profile and progress pages.

Target requirement(s): FR10, FR11, FR18, FR19, FR21, FR22

- **Chart generation:**
  1. Ensures repetition and exercise chart data are generated correctly from activity records.
  2. Verifies chart functions return safe default values when no data exists or when Firestore fails.
- **Comment retrieval and posting:**
  1. Ensures comments can be retrieved and stored.
  2. Verifies empty comments and Firestore errors are handled appropriately.
- **Name lookup:**
  1. Ensures user names are returned correctly when present.
  2. Verifies default values are used for unknown or incomplete users.

5.7.3 Selected User State Management Tests

Goal: These tests verify local storage and retrieval of selected-user state used for physiotherapist-patient interactions.

Target requirement(s): FR17, FR18, FR19, FR22

- **Selected user storage:**
  1. Ensures selected user IDs are saved only when valid values are provided.
- **Selected user retrieval:**
  1. Ensures selected user IDs and selected user data are returned correctly when stored.

2. Verifies null is returned when no stored state exists.

- **Selected user clearing:**

1. Ensures stored selected-user state can be removed cleanly.

5.7.4 Physiotherapist Invite Code Retrieval Tests

Goal: These tests verify that the system can retrieve the correct invite code for a physiotherapist user when one exists.

Target requirement(s): FR2, FR3

- **Invite code retrieval:**

1. Ensures a physiotherapist's invite code is returned when valid data exists.
2. Verifies null is returned when no invite code exists, the user is invalid, or lookup fails.

5.8 Exercise Data Module

5.8.1 Exercise Retrieval Tests

Goal: The exercise data module verifies storage and retrieval of user-specific and general exercise data used.

Target requirement(s): FR5, FR18, FR21, FR27

- **User exercise retrieval:**

1. Ensures exercises assigned to a user can be retrieved correctly.
2. Verifies undefined is returned safely when no exercise data exists or when lookup fails.

- **Single exercise retrieval:**

1. Ensures an exercise can be retrieved correctly by exercise identifier.
2. Verifies invalid identifiers and Firestore errors are handled safely.

- **General exercise retrieval:**

1. Ensures the shared exercise library is returned correctly when it exists.
2. Verifies undefined is returned when no general exercise data is available.

- **Selected exercise retrieval:**

1. Ensures selected exercise data can be retrieved for a given user.
2. Verifies missing data and Firestore errors are handled safely.

5.8.2 Exercise Assignment and Modification Tests

Goal: These tests verify adding, updating, and removing user exercises as part of the physiotherapist exercise assignment workflows.

Target requirement(s): FR6, FR18, FR21

- **Exercise removal:**

1. Ensures valid user exercises can be removed correctly.
2. Verifies invalid requests and Firestore failures do not perform unintended deletions.

- **Exercise addition:**

1. Ensures valid exercises can be assigned to users and written correctly to storage.
2. Verifies errors prevent incomplete or inconsistent updates.

- **Exercise modification:**

1. Ensures valid exercises can be updated with new data.
2. Verifies invalid inputs and Firestore errors are handled safely.

References

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. Design thinking document. https://github.com/M9Huynh/technically-functional/tree/32182cfc56c602466e270b7fcbad01d892911ea1/docs/Extras/DesignThinking, 2025a.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. Development plan. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/DevelopmentPlan/DevelopmentPlan.pdf, 2025b.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. Hazard analysis. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/HazardAnalysis/HazardAnalysis.pdf, 2025c.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. Problem statement and goals. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/ProblemStatementAndGoals/ProblemStatement.pdf, 2025d.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. System requirements specification. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/SRS/SRS.pdf, 2025e.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shanthamoorthy, and Maham Siddiqui. Usabilitytesting. https://github.com/M9Huynh/technically-functional/tree/cc8232d8f631d2301785c3421aeb9c8e9f3d9d66/docs/Extras/UsabilityTesting, 2025f.

SecureFlag. Sensitive data protection in today's software, February 2026. URL https://blog.secureflag.com/2026/02/20/sensitive-data-protection-todays-software/. Accessed: 2026-04-02.

6 Appendix

Additional project information to expand on the previous sections.

6.1 Usability Survey Questions

Please refer to the usability testing folder to find the updated survey questions and the other corresponding documents under `docs/Extras/UsabilityTesting`.

6.2 Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Group Reflection Response:

3. To successfully complete the verification and validation of our project, the team collectively needs to acquire the following knowledge and skills:

    - **Dynamic Testing (Eman):** Knowledge of unit testing, test case design, edge case identification, and ensuring full module coverage.
    - **System Testing (Maham):** Ability to design and evaluate end-to-end system tests that validate overall system behaviour and integration.
    - **Testing Tools and Frameworks (Cieran):** Experience using tools such as pytest, linters (e.g., pylint), and developing structured test suites.
    - **Computer Vision Validation (Vaisnavi):** Understanding of OpenCV and MediaPipe for pose detection, landmark tracking, and validating motion analysis.
    - **Usability and Stakeholder Testing (Matthew):** Skills in interpreting user needs, conducting usability testing, and validating the system against stakeholder expectations.

Individual Reflection Responses:

Vaisnavi - Reflection

1. I think one main thing that went well while writing this deliverable was our communication as a group and how that played into the division of work. We had two sub-teams where Matthew and I were focusing on the POC, while Cieran, Maham and Eman were focused on this VnV deliverable. Our communication throughout aided in making this process of working on the deliverable seamless as everyone seemed to be very looped in on what each sub-team was working on. We had our

weekly meetings to touch base and ask for any help if it was needed, and it worked very well.

2. I think one of the main pain points we came across was as a result of us working on the POC, it was a little bit hard at the start to fully keep up with updates for the VnV Plan. However, as mentioned above, with sharing our progress and updates during our team meetings, this helped in providing more clarity and consistency between all group members, and it allowed everyone to be on the same page.

3. **Please refer to the reflection response above for this group question**

4. To improve the knowledge of OpenCV and MediaPipe pose detection for accurate testing of body landmark tracing, we could do the two approaches:

   (a) Follow documentation and existing examples to understand how to properly utilize OpenCV and Mediapipe to accurately detect the varying body landmarks through pose detection.

   (b) Try to work with small test cases and work on small files to test it out on different body landmarks and see how it works.

Maham - Reflection

1. The team was split into two groups: one was responsible for the POC demonstration, and the other was completing the VnV report. All members were in constant communication even though they had other (very) important deadlines to meet. The sub-teams functioned very well, because each team had members who were (somewhat) more comfortable in the area they were responsible for, and others that wanted to expand their skillset. There was a good balance present. In the end, all members delivered work of the highest calibre and I am happy to be a part of such a great team.

2. I do not think it was a pain point but I had underestimated the amount of thought and importance that my portion would entail. I think it was challenging balancing the high priority of this deliverable (my part) and my other important (health) engagements as well. However, I used all

the time management skills that I have learned and effectively managed to chunk my work into more realizable chunks while ensuring I am taking care of my well-being and delivered some good quality work.

3. **Please refer to the reflection response above for this group question**

4. I had previously acquired some knowledge on software testing and have always been fascinated by it. But to really master the application, I had to be very detail oriented. I did a deep dive into the various components of our system. However, I realized that I had to be careful and not make my focus too narrow - because I was developing system tests. I had to look at the overall picture. I did this in 2 ways. Firstly, I researched 'system testing' to further increase my knowledge and get a more concrete idea. However, since I had prior knowledge, I found most of the resources online just broke down system testing into different unit tests. I did not want to pursue that because Eman was working on the unit testing component. I spoke to my teammates on the possible overlap between unit and system testing, and was more confident in my (potential) analysis and approach. In the end, I refined my testing knowledge and by designing different test cases for any possible scenario, and making sure the system was approached from various aspects without being too focused on individual modules.

Matthew - Reflection

1. I think one thing that went well when writing this deliverable was how we divided these two weeks up. We had taken the advice from the professor to begin development on the POC while working on the VnV plan. The team was divided into myself and Vais on the POC while Cieran, Maham and Eman were working on the VnV plan. This allowed us to do brief check-ins with each other while beginning development and thus more time to develop before the POC demo.

2. I think some pain points during this deliverable was tied to external workloads. Since half of the deliverable encompassed reading week, this meant that the team had midterms and assignments for other classes due before this deliverable and managing that was more difficult. We resolved this by being transparent with each of our workloads, by letting

each other know in our Teams chat if we were going to be busy and when the next time we would be available is. This allowed us to plan the deliverable better and achieve internal deadlines.

3. **Please refer to the reflection response above for this group question**

4. I would like to improve my knowledge of stakeholder understanding. I think that with stakeholder understanding it is important as they are the target audience for the application and their needs are to be met even if it differs from our own since they are the target. My approach for that will consist of understanding interviews as well as conversations with the stakeholder during testing. Another approach will be different user groups to see if our initial persona and ideas for the stakeholder would make sense.
   I think the is important since it is easy to develop requirements by making assumptions but to bridge the gap to the target audience is crucial to make a product or service that they would actually use.

Cieran - Reflection

1. Team communication as always but I won't expand further on that. There was a decision to split the team into two subteams; a proof of concept team to begin the implementation and a verification and validation team to tackle this document. I feel the subteams have both been incredibly efficient in their work and the communication between the two has not faltered. The proof of concept team worked well together in overall and in subteam meetings to grasp an understanding of the software packages we will be utilizing while the verification and validation team ensured the entire system was covered within the tests outlined in this document.

2. I expanded on it during the last deliverable, that each person had this 'own version' of the system in their mind and we were all building towards a different thing when the assignment started. That wasn't entirely gone here and we hit a few bumps worrying about what each of the others was including within the system while thinking we don't need that to be included. Through team meetings and a general walkthrough of the flow of the program, we all got on the same page and managed to

generate tests and feel as if we've properly outlined a system to ensure the system is designed properly.

3. **Please refer to the reflection response above for this group question**

4. I feel my personal interaction with test suite development lacks for the requirements of this project. To advance this I will attempt to do the following:

   (a) Follow online guides, walkthroughs, and documentation on the software packages (Python Extension, pytest, pylint, etc.) to ensure the software testing suite developed contains both meaningful tests derived from this document while also covering all the software.

   (b) Generate and execute tests in advancing complexity to ensure an understanding of the routines gone through with a testing suite. 'Getting my hands dirty'.

Eman - Reflection

1. The team worked together effectively by dividing into two focused groups, one for the POC demonstration and another for the VnV plan. Communication was strong throughout, and everyone contributed their strengths while helping others learn. The collaboration created a good balance where team members could build on their existing skills while developing new ones.

2. My main challenge was ensuring complete test coverage across all system modules. I initially underestimated how much detail was needed to properly test each component in isolation. Identifying all the edge cases and understanding how to structure effective unit tests took more effort than expected. I resolved this by creating a systematic checklist that mapped each module to its requirements, then worked through them methodically. Collaborating with teammates to review my test cases also helped catch gaps I had missed.

3. **Please refer to the reflection response above for this group question**

4. I focused on mastering unit testing and test coverage. My first approach was working through practical examples to understand fixtures, mocking, and test isolation. My second approach was carefully reviewing our system requirements and architecture to understand what each module should do. I chose this combination because I needed both the technical testing skills and domain knowledge about our system. Understanding the requirements helped me design tests that actually validated meaningful functionality rather than just checking basic operations. This dual approach allowed me to create comprehensive unit tests covering all eleven modules in our system.